Adapting Language Models

CSCI 601-471/671 (NLP: Self-Supervised Models)

https://self-supervised.cs.jhu.edu/sp2026/

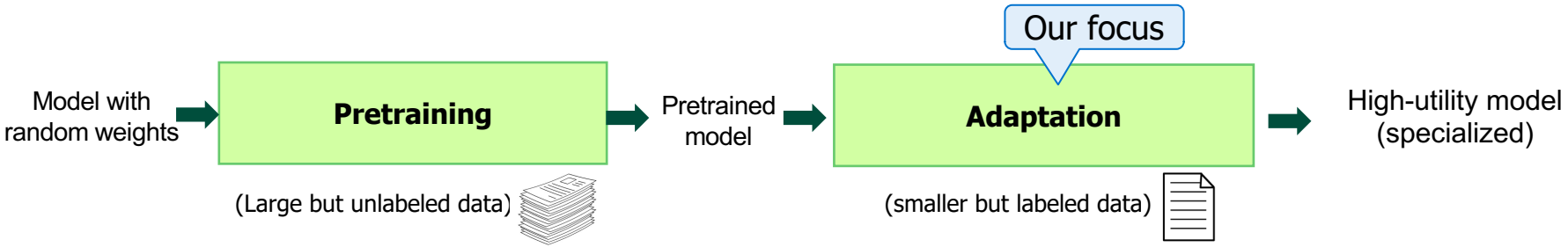
Adapting Language Models: Chapter Plan

You have a pre-trained language model that is pre-trained on massive amounts of data.

They do not necessarily do useful things—they only complete sentences.

Now how to you ”adapt” them for your use-case?

- **Prompting:** adapting model inputs (language statements)
- **Tuning:** adapting (modifying) model parameters



Adaptation: Problem Setup

- Given a pre-trained LM, we want to adapt it to our input-output pairs.

task $\mathcal{T} : \mathcal{X} \rightarrow \mathcal{Y}$

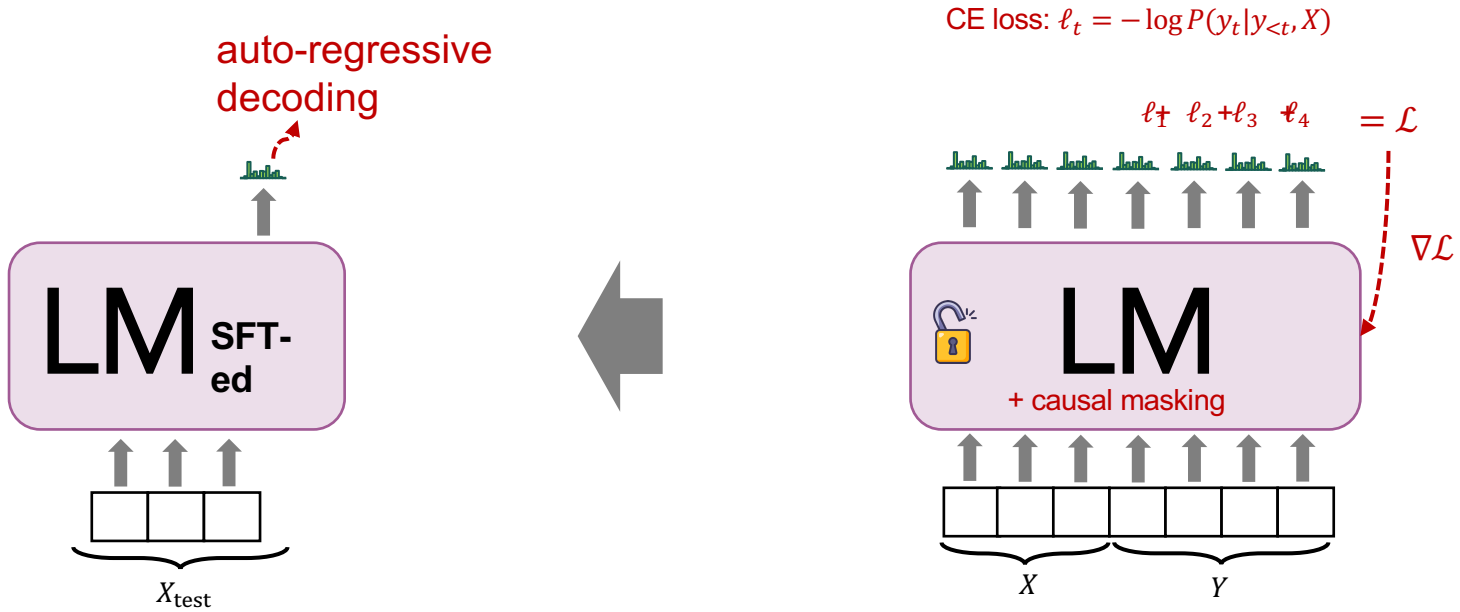
N labeled data

$$\begin{cases} X_1 \rightarrow Y_1 \\ \vdots \\ X_N \rightarrow Y_N \end{cases}$$

LM

Adaptation via Fine-Tuning

Supervised Fine-tuning (SFT)



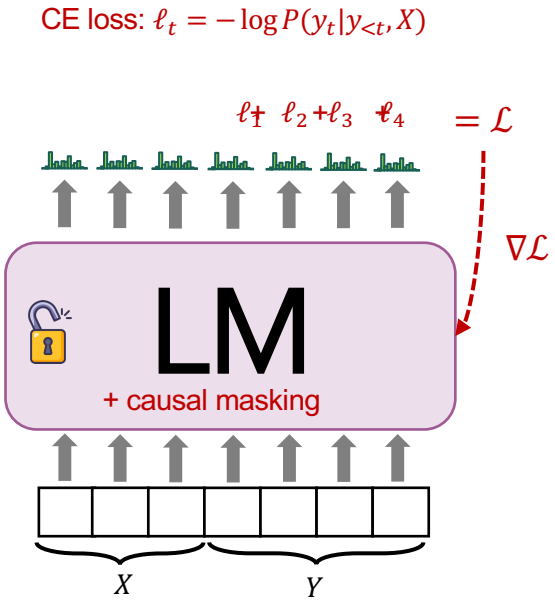
Test time

Training time

SFT on Full model vs Head

- **Whole model tuning:**
  - Your training (optimization) updates **all** model parameters

- **Head-tuning:**
  - Your training (optimization) updates the parameters of the model “head” (the unembed matrix)



Training time

SFT on Full model vs Head: Pros and Cons

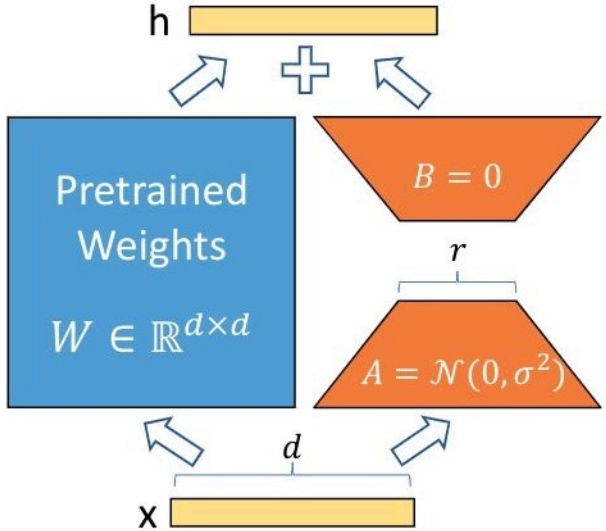
- In full-model SFT we update and store **all** the parameters
  - Need to store **a copy of all the LM for each task**

- **Storage space:** With large models, storage management becomes difficult
  - E.g., A model of size 170B parameters requires ~340Gb of storage
  - If you fine-tune a separate model for 100 tasks:
    - 340 x 100 = 34 TB of storage!
  - Head-tuning requires both less storage

- **GPU memory:** The more *trainable* parameters you have, the more GPU memory need.
  - Full-model training needs more GPU memory for training.
  - Head-tuning requires both less storage and less GPU memory. (even though you effectively keep all model parameters in GPU in both cases).

LoRA: Low-Rank Adaptation

- Hypothesis: the intrinsic rank of the weight matrices in a large language model is low.
- Parameter update for a weight matrix is decomposed into a product of two low-rank matrices.



$$W \leftarrow W + \Delta W$$

$$\Delta W = BA$$

$$B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times d}, \qquad r \ll d$$

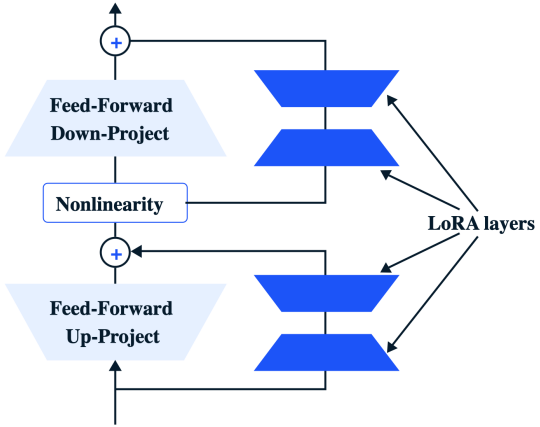
- A is initialized with random Gaussian Initialization, B is initialized to zero.

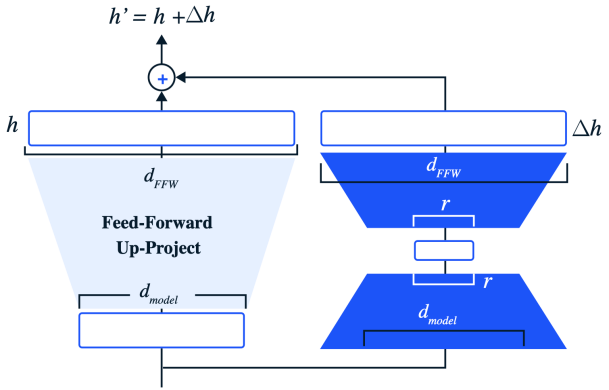
"LoRA: Low-Rank Adaptation of Large Language Models" Hu et al., 2021.

LoRA: Where you place them?

- LoRA used to be applied to W_q and W_v matrices in SA.
- But since then, it's been common to target all matrices in SA and FFN of all layers.

$$\{W_q^{(l)}, W_k^{(l)}, W_v^{(l)}, W_o^{(l)}\}_{l=1}^{L} \quad \{W_{\text{gate}}^{(l)}, W_{\text{up}}^{(l)}, W_{\text{down}}^{(l)}\}_{l=1}^{L}$$



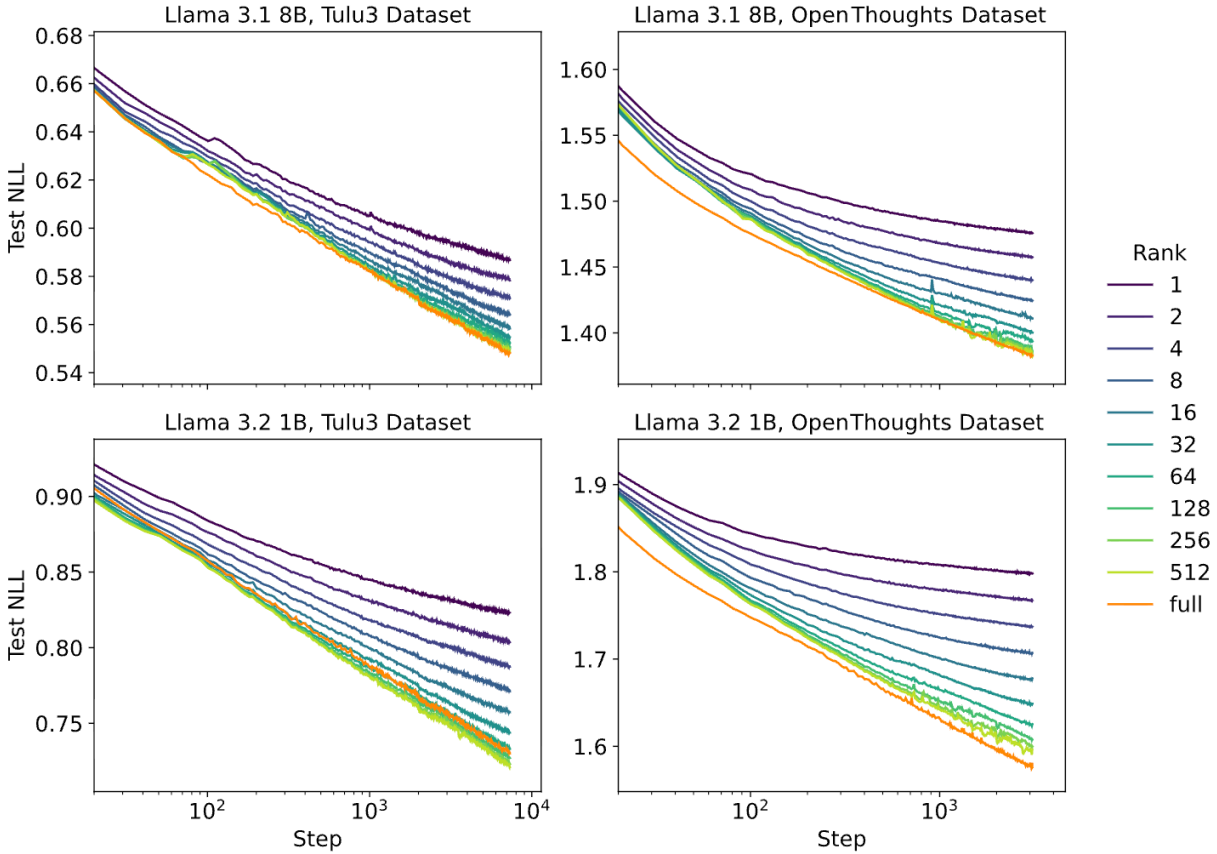


This blog post shows that LoRA on all parameters is more effective:
[LoRA Learns Less and Forgets Less, 2024]

[Figure credit: Samet Oymak]

LoRA, empirically

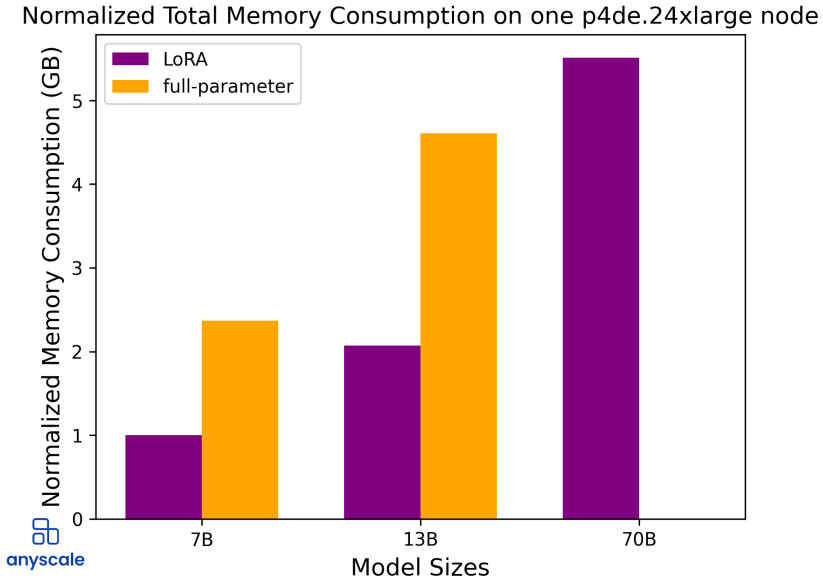
- LoRA [at low ranks] underperforms full finetuning.
- High-rank LoRA can perform (almost) as good as FullSFT



LoRA Learns Less and Forgets Less, 2024

LoRA memory footprint

- LoRA reduces memory requirements during fine-tuning (up to x3 for LoRA).
- Why?



https://www.anyscale.com/blog/fine-tuning-llms-lora-or-full-parameter-an-in-depth-analysis-with-llama-2

LoRA: Compute Cost

- Does LoRA lead to computational speedup?

- **No, in terms of FLOPs:** LoRA does not reduce the number of floating-point operations during the forward or backward pass.
  - The full weight matrix $W + BA$ is still computed, so per-step compute is unchanged (or marginally higher).

- **Yes, in terms of memory efficiency:** LoRA dramatically reduces the number of *trainable* parameters.
  - Since optimizer states (e.g., Adam's first and second moments) are only maintained for *trainable* parameters, GPU memory consumption drops significantly.
  - This allows larger batch sizes, which allows more tokens processed per second, even though per-step latency is not reduced.

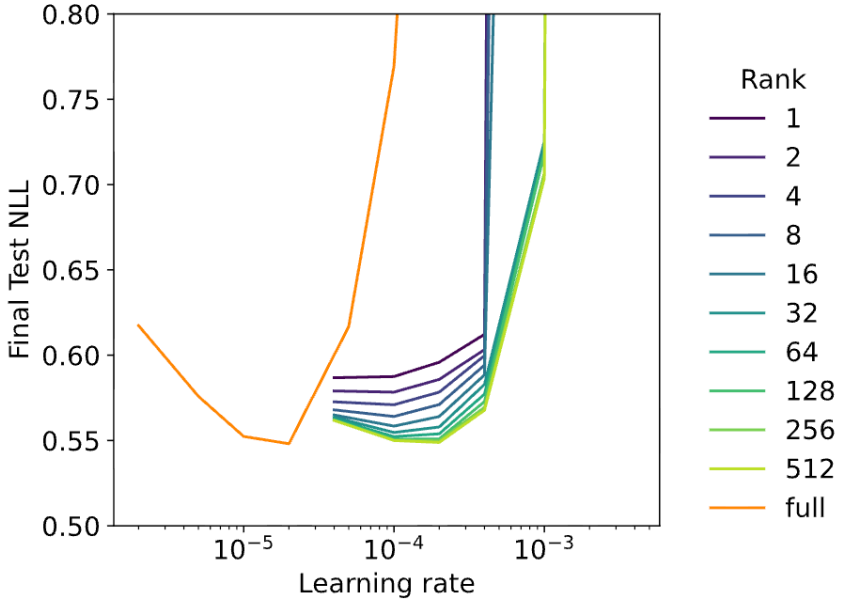
Recap: Model Adaptation

- SFT: A gradient-based way of adapting models for for a given training objective.
- LoRA: Low-rank adaptation to Transformer weight matrices.
- Memory footprint: LoRa reduces memory requirements during fine-tuning (up to x3 for LoRA).
- The main speedup is coming from the fast that you can pack more things in a batch.

LoRA, empirically: Selecting Learning Rate

- Optimal learning rate for LoRA is 10x larger than for full model SFT.

- This is a more general principle: the larger your trainable parameters are, the lower learning rate you need (with all else equal).

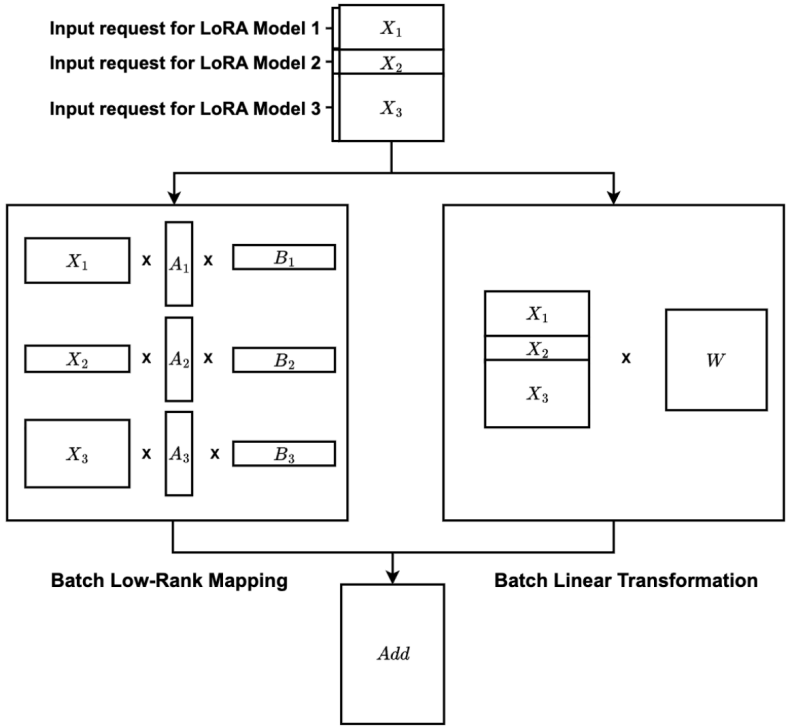


LoRA Learns Less and Forgets Less, 2024

Other benefits of LoRA

- **Multi-tenant serving.**
  - A single inference server can keep many adapters (different model versions) in memory and sample from them simultaneously in a batched way.
  - Modern inference engines e.g., vLLM and SGLang implement this feature.

- This will be an effective tool later when we use do RL.



[Punica: Multi-Tenant LoRA Serving, 2023]
[MixLoRA: An Efficient Multi-Tenant Framework, 2025]

LoRA vs Head-Tuning

- Which has more trainable parameters?
  - (remember optimizer states scale w/ trainable params)
  - Let's do the math for Llama-2 7B; $d = 4096$, 32 layers, $V = 32k$
- Assuming we're doing head-tuning with text generation, we train the unembed matrix.
  - $d \times V$ params = 131M parameters
- For LoRA (applied to Q, K, V, O) matrices, let's consider various ranks
  - If $r = 1 \Rightarrow 1 \times 4 \times 2rd = 1 \times 4 \times 2 \times 4096 = 32k$ params
  - If $r = 4 \Rightarrow 4 \times 4 \times 2rd = 4 \times 4 \times 2 \times 4096 = 131k$ params
  - If $r = 16 \Rightarrow 8 \times 4 \times 2rd = 8 \times 4 \times 2 \times 4096 = 0.524M$ params

- For low-ranks, LoRA has less # of trainable parameters (and hence, smaller optimizer states).

LoRA vs Head-Tuning

- For low-ranks, LoRA has less # of trainable parameters ➔ hence, smaller number of optimizer states → hence, LoRA often takes up less space in your GPUs.
  - But again, you want to be careful about the dimensions of your model.

- Which is more performant?
  - The evidence is mixed on this.
  - Depends on various parameters:
    - task type and data distribution, LoRA rank, amount of data, etc.

- In practice, you need to try it for your setup to find out which works better.

- See other adaptation algorithms: https://huggingface.co/docs/peft/

Summary

- TBD